

Indian Institute of Technology Dharwad
CS201L: Artificial Intelligence Laboratory
Lab 5: Dimensionality Reduction and
Distance-Based Classification using PCA and KNN
05-02-2026

Problem Statement

You are given a real-world multiclass classification dataset as a CSV file named `activity-detection.csv`, constructed by combining the original training and testing files of the **Human Activity Recognition (HAR) using Smartphones Dataset**.

The objective is to classify human activities into one of six daily activities based on sensor-based features extracted from smartphone inertial signals.

Dataset Description

The Human Activity Recognition database was built from recordings of 30 participants performing activities of daily living (ADL) while carrying a waist-mounted smartphone with embedded inertial sensors.

Each subject performed six activities:

- WALKING
- WALKING_UPSTAIRS
- WALKING_DOWNSTAIRS
- SITTING
- STANDING
- LAYING

The smartphone (Samsung Galaxy S II) captured:

- 3-axial linear acceleration (accelerometer)
- 3-axial angular velocity (gyroscope)

at a constant sampling rate of 50Hz. The signals were preprocessed using noise filtering and segmented into fixed-width sliding windows of 2.56 seconds with 50% overlap (128 readings per window).

From each window, a vector of features was extracted from both time and frequency domains.

Dataset Properties

- Total subjects: 30
- Activities: 6 (multiclass)
- Total features: 561 (numerical)
- Feature domains: Time-domain and Frequency-domain
- Sensors: Accelerometer, Gyroscope

- Target attribute: **Activity**
- Data format: CSV file
- Dataset file: `activity-detection.csv`

Attribute Information

For each record in the dataset, the following information is provided:

- Triaxial acceleration from the accelerometer (total acceleration)
- Triaxial body acceleration (body motion component)
- Triaxial angular velocity from the gyroscope
- Gravity acceleration components
- 561-feature vector extracted from time and frequency domains
- Activity label (multiclass target)
- Subject identifier (ID of participant)

Dataset Statistics

Property	Value
Total Samples	10,299
Total Features	561
Target Classes	6
Missing Values	0
Data Types	float64 (561), int64 (1), object (1)

Table 1: Overall Dataset Information

Activity	Sample Count	Percentage (%)
LAYING	1,944	18.88
STANDING	1,906	18.51
SITTING	1,777	17.25
WALKING	1,722	16.72
WALKING_UPSTAIRS	1,544	14.99
WALKING_DOWNSTAIRS	1,406	13.65
Total	10,299	100.00

Table 2: Activity-wise Sample Distribution

Tasks

Write a Python program to complete the following tasks.

Part A: Data Preparation, Standardization, and PCA

1. Load the dataset
 2. Separate features (X) and target (y).
 3. Split into training (60%), validation (20%), and testing (20%) using stratified class wise sampling with `random_state=42`.
 4. Save the splits as:
 - `ActivityTrain.csv`
 - `ActivityValidation.csv`
 - `ActivityTest.csv`
 5. Standardize the training data using `StandardScaler` and save as `ActivityScaledTrain.csv`.
 6. Use training statistics to standardize validation and test data. Save as:
 - `ActivityScaledValidation.csv`
 - `ActivityScaledTest.csv`
 7. Perform PCA with all components on original training data, then transform validation and test data. Save as:
 - `ActivityPCAAllTrain.csv`
 - `ActivityPCAAllValidation.csv`
 - `ActivityPCAAllTest.csv`
 8. Perform PCA using 95% variance approach to obtain reduced dimensional representation on original training data, then transform validation and test data. Additionally, compute and plot:
 - Eigenvalues in descending order
 - Cumulative explained variance curve
- Save the transformed datasets as:
- `ActivityPCA95Train.csv`
 - `ActivityPCA95Validation.csv`
 - `ActivityPCA95Test.csv`

Note: Include the activity class attribute in all saved CSV files.

Part B: KNN on Original Data

1. Load training, validation, and test data from generated CSV files.
2. Apply KNN with $K = 1, 5, 7, 9, 11, 13$ on **validation data**.
3. For each K, compute on validation data:
 - Accuracy
 - Precision (macro, micro)
 - Recall (macro, micro)
 - F1-score (macro, micro)
4. Plot Accuracy vs. K for validation data.

5. Select the best K based on validation accuracy.
6. Using the best K , evaluate on **test data**. Report:
 - Confusion matrix
 - Accuracy, Precision (macro, micro), Recall (macro, micro), F1-score (macro, micro)

Part C: KNN on Standardized Data

1. Load standardized training, validation, and test data.
2. Apply KNN with $K = 1, 5, 7, 9, 11, 13$ on validation data.
3. Compute all metrics for each K .
4. Plot Accuracy vs. K .
5. Select best K based on validation accuracy.
6. Evaluate on test data and report all metrics.
7. Compare results with Part B.

Part D: KNN on PCA (All Components)

1. Load PCA-transformed datasets (all components).
2. Apply KNN with $K = 1, 5, 7, 9, 11, 13$ on validation data.
3. Compute metrics for each K .
4. Plot Accuracy vs. K .
5. Select best K based on validation accuracy.
6. Evaluate on test data and report all metrics.
7. Compare results with Parts B and C.

Part E: KNN on PCA (95% Variance)

1. Load PCA95-transformed datasets.
2. Apply KNN with $K = 1, 5, 7, 9, 11, 13$ on validation data.
3. Compute metrics for each K .
4. Plot Accuracy vs. K .
5. Select best K based on validation accuracy.
6. Evaluate on test data and report all metrics.
7. Report number of components retained after 95% variance threshold.
8. Compare test results with all previous parts and analyze dimensionality reduction impact.

Additional Question

Part F: KNN on PCA (99% Variance)

1. Perform PCA using 99% variance approach to obtain reduced dimensional representation on original training data, then transform validation and test data. Save as:
 - ActivityPCA99Train.csv
 - ActivityPCA99Validation.csv

- ActivityPCA99Test.csv
2. Load PCA99-transformed datasets.
 3. Apply KNN with $K = 1, 5, 7, 9, 11, 13$ on validation data.
 4. Compute metrics for each K .
 5. Plot Accuracy vs. K .
 6. Select best K based on validation accuracy.
 7. Evaluate on test data and report all metrics.
 8. Report number of components retained after 99% variance threshold.
 9. Compare test results with Part E (95% variance) and analyze the impact of retaining 99% variance versus 95% variance on:
 - Number of components retained
 - Classification performance
 - Dimensionality reduction effectiveness

Important Notes

1. For train-validation-test split (60%-20%-20%), use:

```
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.4, stratify=y, random_state=42)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.5, stratify=y_temp,
    random_state=42)
```

Keep `random_state=42` for all splits to ensure consistency.

2. For PCA with variance threshold:
 - 95% variance: `PCA(n_components=0.95)`
 - 99% variance: `PCA(n_components=0.99)`

3. Import required libraries:

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import (accuracy_score, precision_score,
                             recall_score, f1_score, confusion_matrix)
```

4. For multiclass metrics, use `average='macro'` and `average='micro'`.
5. Required plots (6 total):

- Cumulative explained variance with 95% threshold (Part A)
- Accuracy vs. K on validation data for original data (Part B)
- Accuracy vs. K on validation data for standardized data (Part C)
- Accuracy vs. K on validation data for PCA (all components) (Part D)
- Accuracy vs. K on validation data for PCA (95% variance) (Part E)
- Accuracy vs. K on validation data for PCA (99% variance) (Part F)

You may combine the five Accuracy vs. K plots into a single comparative plot.

6. For each part (B, C, D, E, F):

- Use validation data to find the best K
- Report validation metrics for all K values
- Report final test metrics only for the best K

Performance Metrics

- **Accuracy:** Overall correctness
- **Precision (Micro):** Global calculation across all classes
- **Precision (Macro):** Unweighted mean per class
- **Recall (Micro):** Global calculation across all classes
- **Recall (Macro):** Unweighted mean per class
- **F1-score (Micro):** Harmonic mean of micro precision and recall
- **F1-score (Macro):** Unweighted mean per class

— End of Lab Manual —